

call(), apply(), and bind() in JS

1. What is call(), apply(), and bind() in JS?

- JavaScript provides three methods for manipulating *'this'* keyword in functions: `call()` , `apply()` , and `bind()` .
- These methods allow you to change the context of this keyword, which can be useful for controlling the behaviour of functions.
- The `call()`, `apply()`, and `bind()`, will help us to set explicitly the value of *this* keyword inside the function.
- They will help in method sharing.

2. Why do we need it?

- We need `call()` , `apply()` , and `bind()` in JavaScript to control the value of this in a function, allowing us to use or reuse functions with different contexts.

3. *call()* Method:

- The `call()` method allows you to call a function with a specified *this* value and arguments provided individually.
- The first argument to `call()` sets the this value this value for the targeted object or function being called, and the remaining arguments are passed to the function as arguments.

c. Syntax →



```
methodName.call(object, arg1, arg2, .....)
```

4. **apply()** Method:

- a. The apply() method is similar to the call() method, but it takes an **array of arguments** instead of individual arguments.
- b. The first argument to apply() sets the this value for the targeted object or function being called, and the second argument is an array of arguments to pass to the function.
- c. **Syntax** →



```
methodName.apply(object, [arg1, arg2, .....])
```

5. **bind()** Method:

- a. The bind() method returns a new function with a specified this value and any arguments that are passed to it.
- b. The bind() method does not call the function immediately but instead returns a new function that can be called later.
- c. The returned function or method will work only for a particular one object.
- d. **Syntax** →



```
methodName.bind(object)
```

6. **Advantages of call(), apply(), and bind():**

- a. Explicitly setting the this keyword:
 - i. In JavaScript, the value of this keyword is determined by how a function is called.
 - ii. By using the call() , apply() , and the value of the bind() methods, you can explicitly set this keyword, regardless of how the function is called.

- iii. This can be especially useful when working with complex code structures, where the value of this may change depending on how functions are nested and called.
- iv.

7. Difference Between call(), apply(), and bind():

Difference Between call(), apply(), and bind()			
	Call()	Apply()	Bind()
Execution	At the time of binding	At the time of binding	At the time when we execute the return function
Parameter	Any number of arguments one by one	Array[]	Array and any number of arguments
Is Return Function	Yes, it returns and calls the same function at that time of binding.	Yes, it returns and calls the same function at that time of binding	Yes, it returns a new function or copy of the function. Which we can use whenever we want. It's like a loaded gun.